

On-Chip Memory Management Module

Final-Evaluation Answer Sheet

Research problem, gap, novelty, contribution, artefacts, methodology,
scheme comparison, results, limitations and validation

Memory Subsystem — QuadraMind DNN Accelerator Simulator

7 August 2026

One-paragraph summary — if you get 60 seconds, say this

A DNN layer’s working set does not fit in on-chip SRAM, so tiles must be staged in and out of a scratchpad. Conventional accelerators do this with a *runtime* structure — a page table or a cache tag array — which costs a lookup on every access, fetches at page granularity, and places data at addresses the compiler cannot predict, so it cannot avoid bank conflicts. My module implements the alternative: because a DNN schedule is fully known at compile time, a compiler can pre-compute every scratchpad layout and emit only the *differences* between consecutive layouts as a five-opcode delta stream, so no runtime tag or translation structure is needed at all. I built that compiler, the hardware that executes its stream, a banked scratchpad with cycle-exact conflict counters, and a page-table scheme to compare against — and I measured both on identical RTL. Result: 73.2 % less off-chip read traffic than always-reload, and 42× fewer bank conflicts than the paged scheme on the same layer. I also report where paged wins, and why.

Contents

1 Research problem

1.1 The problem in one sentence

Research problem

For a systolic-array DNN accelerator, **how should the limited on-chip scratchpad be managed** — who decides which tensor tiles occupy which physical addresses, and when — so that off-chip DRAM traffic, address-translation overhead and SRAM bank conflicts are all minimised simultaneously?

1.2 Why it matters — the quantitative motivation

A single convolution layer with $C=16$ input channels, $K=32$ output channels, a 14×14 feature map and 3×3 kernels needs

$$\underbrace{C H W}_{\text{inputs}} + \underbrace{K C K_h K_w}_{\text{weights}} + \underbrace{K O_h O_w}_{\text{outputs}} = 3136 + 4608 + 6272 = 14,016 \text{ elements}$$

which at 4 bytes/element is **56 KB against a 16 KB scratchpad**. The tensors must be tiled, and the accelerator must continuously decide what stays resident. That single decision drives three costs:

1. **Off-chip traffic.** A DRAM byte costs roughly two orders of magnitude more energy than an on-chip SRAM byte (≈ 560 pJ/B versus a few pJ). Redundant re-fetches dominate the energy budget.

2. **Address-translation overhead.** Any runtime structure mapping logical tile addresses to physical scratchpad locations charges a lookup on *every* access, replicated per read port.
3. **Bank conflicts.** The scratchpad is banked so several ports can read per cycle. When two ports hit the same bank they serialise and the array stalls.

1.3 The sub-questions this module answers

1. Can a compiler eliminate the runtime tag/translation structure entirely, and what does that cost instead?
2. How much off-chip read traffic does compile-time delta fetching avoid, across layer shapes, scratchpad capacities and array sizes?
3. Static versus dynamic management — what is the real trade in area, clock frequency, cycles, traffic and conflicts?
4. Are bank conflicts a random phenomenon (in which case nothing can be done) or a deterministic function of the access pattern (in which case compile-time placement can control them)?

2 Research gap

Research gap

No existing DNN-accelerator simulator models off-chip traffic, address-translation cost and bank conflicts *cycle-accurately in the same framework*, and none of them supports a compiler-assisted tagless management scheme at all. Every published comparison of memory-management schemes for accelerators is therefore either analytical or restricted to cache/page-table variants.

2.1 What the existing tools do and do not do

Simulator	Off-chip traffic	Bank conflicts	Tagless / compiler-manage
SCALE-Sim	analytical	no	no
Timeloop / Acceleergy	analytical	no	no
CiMLoop	analytical	no	no
STONNE	partly cycle-level	no	no
ONNXim	cycle-level	no	no
This module	measured (RTL + AXI counters)	cycle-exact counters	yes, implemented in RTL

2.2 The three specific gaps

1. **Gap in modelling fidelity.** Bank conflicts are almost always either ignored or approximated with a closed-form birthday-problem bound. That bound assumes *uniformly random* addresses; real DNN access patterns are highly structured, so the bound is wrong in both directions — consecutive access is conflict-free (far better than the bound) and stride- B access is fully serialised (far worse). Nobody had measured which one real tile footprints produce.
2. **Gap in the design space.** The literature compares caches, double buffers and scratchpads with *runtime* allocation. The compiler-assisted static tagless point — no tag array, no page table, placement fixed at compile time — is discussed conceptually but has not been implemented in RTL and measured against a dynamic scheme on identical hardware.
3. **Gap in evidence.** Scheme comparisons in accelerator papers typically quote normalised factors from prior work. There was no artefact where the same layer, the same array, the same clock and the same counters produce both schemes' numbers.

Say this if asked “isn’t delta encoding old?”

Yes — delta encoding itself is decades old. The gap is not the idea of a delta; it is (a) applying it to *scratchpad layout* rather than to data values, (b) using it to remove the runtime translation structure completely, and (c) measuring the consequence cycle-accurately against a page table. It is the third that no prior simulator provides.

3 Novelty

The three novel claims

N1 — Delta-of-layouts, not delta-of-data. The compiler computes the difference between consecutive *scratchpad layouts* (which tile sits at which physical address in each phase) and emits it as a five-opcode stream. The unit of the delta is a tile placement, not a data value.

N2 — Tagless by construction. Because a DNN’s loop nest is fully known before execution, there is nothing to *discover* at runtime, so there is nothing for a tag or a page table to record. The tag array, comparators and replacement policy are removed, not optimised.

N3 — Measured, not modelled. Both schemes run on the same RTL, and every reported number comes from a hardware counter, an AXI responder or a Vivado synthesis report — including the numbers that do *not* favour my scheme.

3.1 Supporting novelty

- **The ZERO opcode.** Padded convolutions have receptive fields extending past the tensor border. Those bytes are implicit zeros: they must exist on-chip but must never be fetched from DRAM. Giving them their own opcode — rather than charging them as loads — reduced the reported load volume from 235,008 B to 157,888 B and made the measurement honest rather than flattering.
- **Bank conflicts made a first-class, controllable quantity.** The banked scratchpad exposes cycle-exact conflict and stall-port-cycle counters, and the measured sweep proves conflicts are a deterministic function of the address pattern — which is exactly what compile-time placement controls. This converts “bank conflicts” from an assumed overhead into a design variable.
- **Three-way independent agreement** (compiler / software FSM model / SystemVerilog RTL) as the correctness argument, rather than a single golden vector.

4 My contribution

Concretely, what I built and what I measured.

4.1 Compiler side

- `stamp_compiler.py` (985 lines) — decomposes a conv layer into *phases*, allocates a *stamp* (a complete scratchpad layout) per phase, computes the *delta* between consecutive stamps, and emits three artefacts: `delta_ops.hex` (the hardware op stream), `stamp_phase_table.svh` (phase base addresses) and `stamp_metadata.json` (the machine-readable schedule).
- Classification rules based on tile *identity* rather than shape: weights are reusable only when the output-channel slice matches; input reuse is computed from receptive-field overlap; a simultaneous row-and-column shift conservatively falls back to a full reload, so **measured reuse is a lower bound, never an overstatement**.
- Padding clipping: the region is intersected with the real tensor extent and split into LOAD bytes and ZERO bytes.

- `stamp_analyzer.py` — reuse and bandwidth analysis over the emitted stream.

4.2 Hardware side (RTL)

- `stamp_based_memory_controller.sv` — a nine-state FSM that reads the compiler’s metadata word by word and executes it: issues AXI bursts for LOAD, on-chip read/write pairs for MOVE, nothing for KEEP and ALLOC, and write-only zero fills for ZERO. Exposes `stats_loads`, `stats_moves`, `stats_keeps`, `stats_bytes_loaded`, `stats_bytes_moved` and `stats_bad_ops`.
- `scratchpad_ram.sv` — dual-mode SRAM. `NUM_BANKS==1` is a flat, conflict-free baseline; `NUM_BANKS ≥ 2` is low-order interleaved with per-cycle conflict arbitration (lowest-indexed port wins, losers get `rd_valid=0` and retry) and three monitoring outputs: `bank_conflict_detected`, `stats_bank_conflicts`, `stats_bank_conflict_stall_cycles`.
- `paged_memory_backend.sv` + `paged_table.v` — the dynamic comparison scheme: per-port page tables, VPN→PPN translation, hit/miss counters.
- `mem_backend_wrap.sv` — selects STAMP or PAGED by parameter and propagates all counters to the top level.
- **Plumbing fixes I made so the comparison is real:** `NUM_BANKS` is now a genuine top-level parameter (it was hard-defaulted to 4 and never reached the simulator); the bank counters were dangling at `u_mem` and are now top ports; and external `spad_dbg_*` read ports were added so the memory side can be exercised independently of the array.
- **PAGED page-table write bug (found and fixed).** The per-port PT write was gated on `p == 0` and required `rd_addr[0]` to hold the target VPN, so ports 1–3 could never hit — the paged scheme was structurally crippled. Writes now broadcast to all port tables. **The whole comparison was re-run against this corrected baseline**, i.e. against a paged scheme working as well as it can.

4.3 Model and evaluation side

- `pysim/stamp_ref.py` — an independently written cycle-accurate model of the controller FSM, used both as a cross-check and as the engine for large sweeps where RTL simulation is too slow.
- `scripts/sweep_memory_schemes.py` — 6 layer shapes × 4 scratchpad capacities × 2 array sizes; each grid point is a real compiler run, not an interpolation. 34 of 48 points fit; the other 14 are recorded as capacity failures rather than silently dropped.
- `tb/golden/test_scheme_divergence.py` — the STAMP-vs-PAGED head-to-head on real layers with real tile footprints.
- `tb/golden/test_bank_sweep.py` — the four-pattern × five-bank-count conflict sweep.
- `scripts/run_full_eval.py` → `exp4_memory_management` and `exp4b_stamp_vs_paged` — turn the captured artefacts into the report figures.
- **De-hardcoding.** Following the supervisor’s “don’t hardcode anything” instruction I removed the previous four-scheme comparison entirely (see §??) and replaced it with measured static-vs-dynamic. I also removed the invented constants in the bank conflict experiment (`WORKLOAD_MEM_INTENSITY`, `DATAFLOW_PORT_ACTIVITY`, and the clamps that pinned every conv layer to exactly 0.18) and replaced them with quantities derived from each layer’s own geometry.

5 Artefacts

5.1 Source artefacts (what I wrote)

File	Role
static_hash_and_tagless_memory/stamp_compiler.py	Flows by stamps, deltas; emits hex + phase table + metadata
.../stamp_analyzer.py	Reuse and bandwidth analysis
.../stamp_based_memory_controller.sv	9-state delta-op sequencer
.../tb_stamp_based_memory_controller.sv	13-test directed testbench (48 assertions)
.../paged_table.v, tb_paged_table.v	Page table + its testbench
.../run_vivado_sim.sh	One-command reproduction of the RTL evidence
.../synth/synth_area.tcl	Vivado out-of-context synthesis driver
sim_framework/rtl/memory/scratchpad_ram.sv	Banked SRAM + conflict counters
.../rtl/memory/paged_memory_backend.sv	Dynamic (page-table) backend
.../rtl/memory/stamp_memory_backend.sv	Static (STAMP) backend
.../rtl/memory/mem_backend_wrap.sv	Scheme selector + counter propagation
sim_framework/pysim/stamp_ref.py	Cycle-accurate FSM software model
sim_framework/tb/unit/test_stamp_controller.py	27 software tests
sim_framework/tb/unit/test_bank_conflict.py	17 banking tests
sim_framework/tb/golden/test_scheme_diverge.py	511 MB vs PAGED on real layers
sim_framework/tb/golden/test_bank_sweep.py	Bank-conflict pattern sweep
sim_framework/scripts/sweep_memory_schemes.py	34 point compiler sweep
sim_framework/scripts/run_full_eval.py	exp3, exp4, exp4b figure generation
.../docs/memory_module_theory.pdf	Full theory + architecture document

5.2 Generated artefacts (the evidence)

Artefact	What it holds
static_hash_and_tagless_memory/delta_ops.hex	The compiler's op stream, read by RTL via \$readmemh
.../stamp_phase_table.svh	Phase base addresses, 'included by RTL
.../stamp_metadata.json	Machine-readable schedule (drives the accounting figures)
.../stamp_analysis_summary.txt	Reuse / bandwidth summary of the standalone run
.../synth/work/out/area_results.csv	Vivado OOC synthesis: LUTs, FFs, WNS, Fmax for 11 configurations
sim_framework/results/golden_check/raw/diverge	Real-run RTL counters for STAMP and PAGED (4 files)
.../results/golden_check/raw/bank_sweep_b{1,2,3,4}.csv	Conflict/stall counters per pattern
.../results/exp4_memory_management/sweep_stamp3.csv	34 real compiler runs across the design grid
.../results/exp4_memory_management/capacity_limit4.csv	The 4 grid points that do not fit, with the reason
.../results/exp4_memory_management/area_measured.csv	64 ops of the synthesis table used by the figures
.../results/exp4b_stamp_vs_paged/stamp_vs_paged.csv	The head-to-head comparison table
.../results/exp4b_stamp_vs_paged/bank_conflict1.csv	The 16 rows bank sweep table
.../results/exp4b_stamp_vs_paged/stamp_delta_accounting.csv	The 256 rows zero byte accounting
.../results/vivado/stamp_controller_xsim.log	The xsim console log (48 assertions, 0 failures)
.../results/golden_check/MEMBER2_MEMORY_GOLDEN_CHECK_REPORT.md	The golden-check report with honesty labels

5.3 Figures produced

results/exp4_memory_management/figures/: area_static_vs_dynamic.png, area_scaling.png, banking_area_cost_offchip_reduction_sweep.png, metadata_overhead.png.

results/exp4b_stamp_vs_paged/figures/: `axi_beats.png`, `axi_ar_requests.png`, `bank_conflicts.png`, `stall_port_cycles.png`, `cycle_breakdown.png`, `bank_stall_port_cycles.png`, `bank_reads_served.png`, `stamp_vs_naive_traffic.png`, `delta_op_mix.png`.

6 Methodology

6.1 Overall flow

Step	What happens
1	Compile. <code>stamp_compiler.py</code> takes a layer geometry and a scratchpad size, builds the phase list, allocates a stamp per phase, and computes deltas. Emits <code>delta_ops.hex</code> , <code>stamp_phase_table.svh</code> , <code>stamp_metadata.json</code> .
2	Model. <code>pysim/stamp_ref.py</code> replays that op stream through a cycle-accurate model of the controller FSM and returns cycles, bytes loaded, bytes moved, bytes zeroed, AXI bursts.
3	Simulate (RTL, xsim). The SystemVerilog controller reads the <i>same</i> hex file via <code>\$readmemh</code> and executes all 127 phase transitions / 572 operations under the directed testbench.
4	Simulate (RTL, Verilator + cocotb). The full accelerator top runs a real layer end-to-end, once with <code>MEMORY=STAMP</code> and once with <code>MEMORY=PAGED</code> , at <code>NUM_BANKS</code> $\in \{1, 2, 4, 8, 16\}$, and dumps every hardware counter to JSON. Correctness is checked against a TensorFlow golden reference.
5	Synthesise. Vivado out-of-context synthesis of the stamp controller, the paged manager and the banked scratchpad, swept over each one's own capacity knob, yields LUT/FF/Fmax.
6	Sweep. <code>sweep_memory_schemes.py</code> runs steps 1–2 over the full layer $\times$ capacity $\times$ array grid.
7	Report. <code>run_full_eval.py</code> reads only the artefacts from steps 4–6 and produces the CSVs and figures. It contains no performance constants.

6.2 Experimental configuration

Parameter	Value
Layers (head-to-head)	<code>tiny_cnn</code> layer 0, <code>mnist_cnn</code> layer 0
Dataflow / layout	Output-stationary, 8×8 array, <code>CHANNEL_MAJOR</code>
Scratchpad read ports	4
Bank counts swept	1, 2, 4, 8, 16
Page size (PAGED)	2^{10} words = 4 KB
Metadata depth (STAMP)	256 / 512 / 1024 entries (1024 used for verification)
Standalone compiler run	16 KB scratchpad, 4 B elements, 128 phases
Sweep grid	6 layer shapes $\times \{8, 16, 32, 64\}$ KB $\times \{4 \times 4, 8 \times 8\}$
Tools	Verilator 5.050 + cocotb 2.0.1, Vivado xsim 2025.2, TF 2.21
Correctness tolerance	5 % of full scale (achieved: 0.009–0.013 %)

6.3 The two conflict models, and which is authoritative

Be precise about this in the viva

There are two bank-conflict numbers in the report and they are not the same thing:

- **Experiment 3** uses the analytical birthday-problem bound, $P_{\text{conflict}} = 1 - \prod_{k=0}^{P-1} \frac{B-k}{B}$, scaled by a per-layer memory-active fraction derived from the layer's own MAC count and byte demand. It is an *estimate*, applied across 14 workloads where RTL simulation of every layer would be prohibitive.
- **Experiment 4b** uses *cycle-exact hardware counters* from the banked scratchpad. It is the authority.

The bound assumes uniform random addresses; real patterns are structured, so the measured values differ from the bound — and that difference is itself one of my findings.

7 Alternative memory management schemes — the comparison

7.1 What is compared against what

Three comparison axes, all measured:

Axis	Alternatives	Where measured
Management policy	STAMP (static, tagless, compiler-assisted) vs PAGED (dynamic, hardware page table)	exp4b — RTL counters; exp4 — Vivado area/Fmax
Fetch policy	Delta fetch vs always-load (a tag-based scheme that re-reads every input and weight tile every phase)	exp4 sweep + stamp_delta_accounting.csv
Memory organisation	Banking degree $B \in \{1, 2, 4, 8, 16\}$ ($B=1$ = conflict-free baseline)	bank_conflict_sweep.csv + area_results.csv

7.2 The two schemes, mechanically

	PAGED (dynamic)	STAMP (static)
Who decides placement	Hardware, at runtime	Compiler, ahead of time
Runtime structure	Page table per port; $\text{VPN} \rightarrow \text{PPN} + \text{valid bit}$	None — direct addressing
Lookup cost	Every access, every port	Zero
Fetch granularity	Whole page (4KB) on a fault	Exactly the new bytes
Bookkeeping storage	Table = $\text{NUM_PAGES} \times (\text{PPN width} + 1)$ bits	Metadata RAM = 128 bits per delta op
Bank residency	Unpredictable to the compiler	Chosen by the compiler
Latency determinism	Non-deterministic (faults)	Deterministic

7.3 The five delta operations (STAMP)

Op	Opcode	Condition	Cost
KEEP	0	identical data, identical address	0 cycles, 0 bytes
MOVE	1	identical data, different address	2 cycles/word (rd+wr)
LOAD	2	data not resident on-chip	DRAM burst
ALLOC	3	output slot, written by compute	0 cycles, 0 bytes
ZERO	4	zero-padding halo	1 cycle/word (write only)

Only LOAD costs off-chip bandwidth. The saving is $\sigma = R/(L + R)$ with $R = M + K$ the bytes resolved on-chip.

7.4 Superseded: the four-scheme comparison

If an examiner has seen the older charts

An earlier version of Experiment 4 compared four schemes — Static, Double Buffer, Unified Buffer, Cache-Based. **Every number in it came from a hardcoded dictionary** (MEMSCHEME_MODEL: area 1.18 / cycles 0.76, etc.) and invented formulas of the form $0.90 - 0.28 m_f$. Two problems: nothing was measured, and those three scheme names appear nowhere in `evaluation_requirements.pdf`, which asks for “tradeoffs between static and dynamic memory management schemes”. On the supervisor’s “don’t hardcode anything” instruction (Aug 2026) the whole experiment was deleted and replaced with the measured static-vs-dynamic comparison. The old outputs are retained, clearly labelled, under `results/{cloud,edge}/_superseded_hardcoded_exp4/` so the change is auditable. **Do not present those figures.**

8 The compared values — what they are, how they are captured

8.1 Metric dictionary

Value	What it means	Source / how captured
<code>stats_bytes_loaded</code>	Bytes actually fetched from DRAM by the memory backend	Hardware counter inside <code>stamp_memory_backend</code> , incremented per accepted AXI beat
<code>axi_ar_requests</code>	AXI read-address handshakes issued	Counted by the cocotb AXI responder model
<code>axi_beats</code>	AXI data beats returned	Counted by the cocotb AXI responder model
<code>stats_bank_conflicts</code>	Cycles in which ≥ 1 port was denied its bank	Counter in <code>scratchpad_ram.sv</code> , increments when <code>bank_conflict_detected</code>
<code>stats_bank_conflict_stall_cycles</code>	By a 4-port-cycles stalled (sum over stalled ports)	Same block; adds the stalled-port count each conflict cycle
<code>compute_cycles</code>	Cycles the top FSM spends in compute for the whole layer	Free-running RTL cycle counter, sampled at layer end
<code>program_cycles</code>	Cycles spent writing the meta-data / page table	Same counter, sampled around the programming phase
<code>replay_cycles</code>	Cycles of the 4-port footprint replay	Same counter, sampled around the replay
<code>stats_loads_or_hits</code> , <code>stats_moves_or_misses</code> , <code>stats_keeps</code>	STAMP: load/move/keep op counts. PAGED: page hits/misses	Backend counters; the wrapper reuses the same ports for both schemes
<code>new_words / reused_words</code>	Word-touches classified new vs already resident	Host-side bookkeeping in the testbench, cross-checked against the HW counter
<code>luts</code> , <code>ffs</code> , <code>wns_ns</code> , <code>fmax_mhz</code>	Post-synthesis resource and timing	Vivado out-of-context synthesis reports
<code>reduction_pct</code>	$1 - L/\text{naive}$, off-chip read traffic avoided	Compiler + FSM model over the sweep grid
<code>metadata_bytes</code>	On-chip RAM the delta stream needs	$16\times$ (total delta ops)

8.2 The capture process, step by step

(a) RTL counters for the head-to-head (`divergence_*.json`).

1. `tb/golden/run_golden.py` builds the design with Verilator, passing `-GMEMORY={0,1}` (STAMP/PAGED) and `-GNUM_BANKS=n` as parameter overrides.
2. `test_scheme_divergence.py` walks the layer’s OS tile sequence. For **STAMP** it tracks which DRAM words are already resident and programs the metadata table with LOAD ops for contiguous runs of new words and KEEP ops for contiguous runs of reused words; the hardware engine then executes them during the stamp phase, issuing real AXI bursts. For **PAGED** it replays the tile footprint’s DRAM addresses through the translation hardware, so unmapped pages MISS and mapped pages HIT, then maps the faulted pages with `pt_write`.
3. After each tile, the tile footprint is replayed on all four external `spad_dbg_*` read ports, so the banked arbitration sees real DNN address patterns.
4. At the end, every counter is read out and written to `results/golden_check/raw/divergence_{layer}_{scheme}_` together with the full output tensor and a pass/fail verdict against the TensorFlow reference.

(b) **Bank sweep** (`bank_sweep_b*.json`). `test_bank_sweep.py` never starts the DUT FSM — it measures the memory side alone. It drives the four external read ports with a fixed address pattern for a 200-cycle window and reads the hardware counters before and after:

Pattern	Port <i>p</i> reads address
<code>same_addr</code>	0 (all ports)
<code>consecutive</code>	<i>p</i>
<code>stride4</code>	$4p$
<code>stride8</code>	$8p$

Repeated for $B \in \{1, 2, 4, 8, 16\}$ (five separate builds).

(c) **Area and Fmax** (`area_results.csv`). `synth/synth_area.tcl` runs Vivado out-of-context synthesis on each module in isolation (no I/O buffer insertion, so the numbers are the logic’s own cost), sweeping the stamp controller over `METADATA_DEPTH` $\in \{256, 512, 1024\}$, the paged manager over `NUM_PAGES` $\in \{128, 256, 512\}$ and the scratchpad over `NUM_BANKS` $\in \{1, 2, 4, 8, 16\}$. The utilisation and timing reports are parsed into one CSV row per configuration.

(d) **Compiler sweep** (`sweep_stamp.csv`). `scripts/sweep_memory_schemes.py` instantiates `StampCompiler` at each grid point, calls `create_conv_phases` $\rightarrow$ `allocate_stamps` $\rightarrow$ `compute_deltas`, then replays the emitted ops through `pysim.stamp_ref.run_stream`. If `allocate_stamps` raises (the phase does not fit), the point is recorded in `capacity_limits.csv` instead of being dropped.

(e) **Standalone compiler-to-RTL evidence**. `run_vivado_sim.sh` runs the compiler, runs the analyzer, then compiles and simulates `tb_stamp_based_memory_controller.sv` in `xsim` against the generated `delta_ops.hex`, and copies the console log into `results/vivado/`.

8.3 Reproduction commands

To regenerate	Command
Compiler artefacts + xsim log	<code>./static_hash_and_tagless_memory/run_vivado_sim.sh</code>
Area / Fmax CSV	<code>vivado -mode batch -source synth/synth_area.tcl</code>
Compiler sweep CSVs	<code>PYTHONPATH=. python scripts/sweep_memory_schemes.py</code>
RTL head-to-head + bank sweep	<code>PYTHONPATH=. python tb/golden/run_golden.py -memory {STAMP,PAGED} -num-banks <i>n</i> -layer ...</code>
Software test suites	<code>PYTHONPATH=. python -m pytest tb/unit/test_stamp_controller.py tb/unit/test_bank_conflict.py -v</code>
All figures	<code>PYTHONPATH=. python scripts/run_full_eval.py</code>

9 Comparison results

9.1 R1 — STAMP vs PAGED, measured RTL counters

Whole layer, OS 8×8, CHANNEL_MAJOR, 4 banks. Source: `results/exp4b_stamp_vs_paged/stamp_vs_paged.cs`

Metric	tiny STAMP	tiny PAGED	mnist STAMP	mnist PAGED
Correctness (max rel. err)	PASS 0.013 %	PASS 0.013 %	PASS 0.009 %	PASS 0.009 %
Bank-conflict events	2	84	101	232
Stalled port-cycles	4	240	202	488
Backend off-chip bytes	1,640 (meas.)	8,192 (derived)	3,448 (meas.)	8,192 (derived)
AXI read requests	157	138	945	832
AXI beats	2,246	1,836	11,470	10,608
Compute cycles	6,316	5,802	38,440	36,608
Metadata / PT programming cyc	41	2	527	2
Replay cycles	456	456	2,912	2,912
LOAD / KEEP ops	19 / 22	—	113 / 414	—
Page hits / misses	—	1,470 / 294	—	10,506 / 102
New / reused word-touches	410 / 1,354	410 / 1,354	862 / 9,746	862 / 9,746

Headline

Bank conflicts: 42× fewer (2 vs 84) on tiny, 2.3× fewer on mnist. **Stalled port-cycles:** 60× fewer (4 vs 240) on tiny. **Off-chip volume:** 5.0× less on tiny (1,640 B vs 8,192 B) and 2.4× less on mnist (3,448 B vs 8,192 B). STAMP’s delta mechanism captured **76.8 %** (tiny) and **91.9 %** (mnist) of word-touches as on-chip reuse. **But PAGED transfers fewer AXI beats on these layers** — and I report that.

9.2 R2 — Off-chip traffic vs the always-load baseline

Standalone 128-phase layer, 16 KB scratchpad. Source: `stamp_analysis_summary.txt`, `stamp_delta_accounting.cs`

Quantity	Bytes	Share
Always-load baseline (inputs + weights, every phase)	589,824	100.0 %
STAMP scheme, actual DRAM reads	157,888	26.8 %
Resolved on-chip by MOVE	184,320	—
Resolved on-chip by KEEP	165,888	—
Zero-filled padding (never off-chip)	77,120	—
Off-chip read traffic reduction		73.2 %
Bandwidth saving $\sigma = R/(L+R)$		68.9 %

Op mix: 134 LOAD, 144 MOVE, 72 KEEP, 127 ALLOC, 95 ZERO = 572 operations over 127 phase transitions.

9.3 R3 — The design-space sweep (34 real compiler runs)

Source: `results/exp4_memory_management/sweep_stamp.csv`.

Layer shape	Min red.	Mean red.	Max red.
early-conv 3×3	64.2 %	66.1 %	67.5 %
mid-conv 3×3	59.8 %	67.6 %	72.9 %
late-conv 3×3	55.9 %	62.8 %	66.3 %
pointwise 1×1	32.3 %	36.0 %	37.9 %
wide-spatial	54.9 %	61.1 %	67.3 %
large-kernel 5×5	73.0 %	73.4 %	73.6 %
All 34 points	32.3 %	60.6 %	73.6 %

Controller-cycle saving against a 10-cycle/word DRAM baseline ranges 88.0–95.6 % (mean 91.4 %). 14 of the 48 grid points do not fit and are recorded with the reason in `capacity_limits.csv`.

9.4 R4 — Bank-conflict sensitivity (cycle-exact)

Stalled port-cycles over a 200-cycle window, 4 read ports. Source: `bank_conflict_sweep.csv`.

Access pattern	1 bank	2	4	8	16
All ports $\rightarrow$ same address	0	600	600	600	600
Stride 4	0	600	600	400	0
Stride 8	0	600	600	600	400
Consecutive	0	400	0	0	0

Reads served in the same window: 796 when conflict-free, 199 when fully serialised — a $4\times$ throughput swing driven purely by the address pattern. With *real* traffic (tiny L0, STAMP, whole layer) the conflict / stall counts were 438/876 at $B=2$, 2/4 at $B=4$ and $B=8$, and 0/0 at $B=16$.

9.5 R5 — Area and frequency (Vivado OOC, ZCU104)

Source: `static_hash_and_tagless_memory/synth/work/out/area_results.csv`.

Config	Scheme	LUTs	FFs	WNS (ns)	Fmax (MHz)
stamp_ctrl_m256	STAMP (static)	6,434	22,198	7.533	405.4
stamp_ctrl_m512	STAMP (static)	12,574	43,938	7.237	361.9
stamp_ctrl_m1024	STAMP (static)	24,680	87,468	7.512	401.9
paged_mgmt_p128	PAGED (dynamic)	3,074	2,752	8.361	610.1
paged_mgmt_p256	PAGED (dynamic)	6,110	5,440	8.317	594.2
paged_mgmt_p512	PAGED (dynamic)	12,106	10,816	8.121	532.2
scratchpad_b1	Scratchpad	71,388	8,357	6.341	273.3
scratchpad_b2	Scratchpad	38,186	8,421	8.834	857.6
scratchpad_b4	Scratchpad	44,880	8,421	8.604	716.3
scratchpad_b8	Scratchpad	35,356	8,421	8.875	888.9
scratchpad_b16	Scratchpad	37,713	8,421	9.001	1001.0

At equal capacity (256 entries) STAMP uses $\approx 1.05\times$ the LUTs but $4.1\times$ the registers of PAGED, and clocks 32 % slower (405 vs 594 MHz). Both scale close to linearly with their capacity knob: STAMP 28.6 k $\rightarrow$ 56.5 k $\rightarrow$ 112.1 k LUT+FF over 256 $\rightarrow$ 512 $\rightarrow$ 1024; PAGED 5.8 k $\rightarrow$ 11.6 k $\rightarrow$ 22.9 k over 128 $\rightarrow$ 256 $\rightarrow$ 512.

10 Why the results came out that way

10.1 Why STAMP has $42\times$ fewer bank conflicts

The stamp compiler bump-allocates scratchpad slots, so a tile’s words land in a *dense, contiguous* region. When four ports replay that footprint they read stride-1 addresses, which under low-order interleaving map to four *different* banks — almost conflict-free at $B=4$. The page table preserves page-relative offsets, so a translated `paddr` keeps the original tensor’s row stride; four ports reading four rows of a feature map hit $a, a+W, a+2W, a+3W$, and whenever $W \bmod B = 0$ all four land in the same bank. That is a genuine consequence of each scheme’s placement policy, not an artefact. The bank sweep (R4) is the controlled proof: the same hardware gives 0 stalls for consecutive and 600 for stride-4 at $B=4$.

10.2 Why PAGED transfers fewer AXI beats (the result against me)

A page fault pulls a whole 4KB page. For these small test layers the working set is small enough that a handful of coarse page fetches happens to cover it efficiently, while STAMP issues many small delta bursts and pays more AXI request overhead (157 vs 138 AR handshakes). **My claim is not “fewer bytes always”** — it is fewer conflicts, no translation hardware, and deterministic placement. As layers grow, a page fault pulls far more data than the corresponding delta and the byte comparison inverts; the 73.2% reduction against always-load in R2 is the regime-independent statement.

10.3 Why STAMP burns more compute cycles (6,316 vs 5,802)

Two reasons, and only one is real. (i) STAMP’s real delta-load AXI time is *inside* its compute cycle count (+514 cycles on tiny), whereas the paged backend has no DRAM path in RTL, so its implied page-fetch time is not charged anywhere. (ii) STAMP genuinely pays a metadata-programming cost (41 vs 2 cycles on tiny; 527 vs 2 on mnist). **The cycle comparison is asymmetric in PAGED’s favour and I state so.**

10.4 Why pointwise 1×1 shows the least reduction (32–38%)

Reuse in this scheme comes from receptive-field overlap between consecutive phases. A 1×1 kernel has no overlap — the receptive field is a single pixel, so sliding the window produces an entirely new footprint. Only weight reuse remains. The scheme’s benefit is therefore proportional to kernel extent, which is exactly why large-kernel 5×5 shows the most (73.0–73.6%). This is a property of the workload, not a weakness of the implementation.

10.5 Why the flat scratchpad ($B=1$) costs the most LUTs and the worst Fmax

Counter-intuitive but correct: a flat array serving 4 concurrent read ports must be implemented as replicated/multiplexed distributed RAM, so it burns 71k LUTs and closes at only 273 MHz. Splitting it into banks lets each bank serve one port through a narrow crossbar — 35–45k LUTs and 716–1001 MHz. **Banking is not purely a cost paid for parallelism; on this device it is also cheaper and faster.** The cost of banking is conflicts, not area.

10.6 Why measured conflict counts are far below the birthday bound

For $P=4, B=4$ the analytical bound predicts $P_{\text{conflict}} = 90.6\%$ of cycles. The measured STAMP run shows 2 conflict events across the whole layer. The bound assumes uniformly random addresses; real footprints are consecutive, and consecutive addresses map to distinct banks *by construction*. The gap between 90.6% and ≈ 0 is precisely the value of compile-time placement — and it is why Experiment 3’s analytical numbers must be labelled as an estimate and Experiment 4b’s as the authority.

11 Limitations

1. **Metadata RAM is the scaling limit.** Removing the tag array does not make bookkeeping free — it relocates it to a compile-time instruction stream, $\text{bits} = 128 \times \sum_{\phi} |\Delta_{\phi-1 \rightarrow \phi}|$. For the measured 128-phase layer, $572 \text{ ops} \times 16 \text{ B} = 9,152 \text{ B}$ for a 16 KB scratchpad — about **56 % of the scratchpad size**. Across the sweep grid it ranges 0.7 KB to 51 KB. The default METADATA_DEPTH of 256 entries holds only $\approx 45\%$ of that layer’s stream, so verification uses 1024. **This is the honest headline trade-off.**
2. **Asymmetric off-chip metric.** STAMP’s bytes are measured on the real AXI port; PAGED’s 8,192 B is *derived* (2 measured page faults $\times$ 4 KB) because the paged backend has no DRAM path in RTL. The volume comparison is real; the cycle comparison favours PAGED.
3. **v1 wiring limit — conflicts do not yet back-pressure compute.** All conflicts, stalls, hits and misses are memory-side events. The array currently reads via the `ext_*` ports rather than through the scratchpad, so a stall is counted but does not throttle the pipeline. Full compute-coupling is future work.
4. **Replay stands in for the array’s reads.** The 4-port replays use real tile-footprint addresses and real RTL arbitration, but they model the consumers rather than being them (consequence of item 3).
5. **Delta model handles only pure row or pure column shifts.** A simultaneous row-and-column shift falls back to a full reload, so reported reuse is a **lower bound**.
6. **No eviction or replacement policy in PAGED (v1).** Pages map once and stay. Fine at these footprints ($\ll$ scratchpad); it matters for larger networks and slightly flatters PAGED’s miss count.
7. **Address-unit inconsistency (documented, open).** The stamp datapath is byte-addressed (`dst >> 2` before the scratchpad); the paged datapath passes `paddr` low bits to the word-addressed scratchpad with no shift. The divergence experiment therefore runs PAGED word-addressed with `PAGE_SIZE_BITS=10` for self-consistency.
8. **Two op-stream generators still exist.** The standalone compiler $\rightarrow$ RTL path is fully closed (`delta_ops.hex` read by `$readmemh`), but the integrated cocotb test builds its own load/keep stream rather than consuming that hex. Unifying them is the next step.
9. **Scale of the head-to-head.** Two small layers (`tiny_cnn`, `mnist_cnn`). They were sufficient to demonstrate divergence but are not a full-network evaluation.
10. **Energy is derived**, from a per-byte DRAM constant, not from switching-activity analysis.
11. **FPGA-relative area.** LUT/FF counts are ZCU104 out-of-context synthesis, not ASIC μm^2 ; they are valid for relative comparison between the two schemes, not as absolute silicon area.

12 Future work

1. **Compress the metadata stream.** The op mix is highly regular, so run-length encoding of repeated delta patterns should cut the 128-bit per-op cost substantially. Alternatives: stream the metadata in batches from DRAM, or use a coarser phase granularity.
2. **Close the compute-coupling loop** so a bank-conflict stall actually back-pressures the systolic array, making the conflict count a *performance* number rather than a memory-side event count.
3. **Unify the two op-stream generators** — have the integrated cocotb test consume `delta_ops.hex` directly, closing the last gap between the standalone and integrated paths.

4. **Conflict-aware allocation.** The bank sweep proves conflicts are a deterministic function of the address pattern, so the allocator can be extended to choose base addresses that guarantee concurrently-read operands land in different banks — turning ≈ 0 conflicts from a happy consequence into a guarantee.
5. **Extend the delta model** to simultaneous row-and-column shifts, which would raise reported reuse above the current lower bound.
6. **Add eviction/replacement to PAGED** so the dynamic baseline is realistic at network scale.
7. **Full-network evaluation** (ResNet-50, MobileNet) instead of two small layers, and larger layers where the STAMP-vs-PAGED byte comparison is predicted to invert — a directly falsifiable prediction.
8. **Multi-DNN scratchpad partitioning.** Because placement is static, partitioning between concurrent DNNs is a compile-time decision with no inter-DNN interference in a translation structure — a structural advantage over a shared page table, worth demonstrating.
9. **ASIC synthesis** for absolute area/energy, replacing the derived per-byte DRAM energy constant with switching-activity analysis.

13 How to evaluate the results, and why they are correct

13.1 The correctness argument in one line

Three-way independent agreement

The scheme is implemented **three times, independently** — the Python compiler’s own book-keeping, a separately written cycle-accurate FSM model, and the SystemVerilog hardware — and all three produce **identical** values for every tracked quantity across 572 operations. A disagreement in any cell would mean the compiler is describing hardware that does not exist.

Quantity	Compiler	FSM model	RTL (xsim)	
LOAD operations	134	134	134	identical
MOVE operations	144	144	144	identical
KEEP operations	72	72	72	identical
ALLOC operations	127	127	127	identical
ZERO operations	95	95	95	identical
Bytes from DRAM	157,888	157,888	157,888	identical
Bytes moved on-chip	184,320	184,320	184,320	identical
Bytes zero-filled	77,120	77,120	77,120	identical

The three implementations are `stamp_compiler.py`, `pysim/stamp_ref.py` and `stamp_based_memory_controller.sv`. The RTL reads the compiler’s `delta_ops.hex` directly, replays all 127 phase transitions and 572 operations, and reports `stats_bad_ops = 0` — proving every emitted opcode is one the hardware implements.

13.2 The seven independent validity checks

1. **Functional correctness against TensorFlow.** Every STAMP and PAGED run is checked against a TF 2.21 golden reference with a 5 % of full-scale tolerance. Achieved: **0.013 %** (tiny) and **0.009 %** (mnist) — roughly 400× inside tolerance. A memory scheme that delivered wrong data would fail here first.
2. **Hardware counter vs host bookkeeping.** `stats_bytes_loaded` equals *exactly* 4× the host-computed new-word count (410 words → 1,640 B; 862 → 3,448 B) on both layers. Two independent accountings of the same quantity agree to the byte.

3. **AXI beat conservation.** Total AXI beats = the layout prefetcher’s background traffic (identical in both schemes: 1,836 / 10,608) **plus exactly** the delta-load beats (410 / 862) in the STAMP runs. The paged backend issues zero AXI by design. Nothing is unaccounted for.
4. **Page-table conservation.** PAGED hits + misses = exactly the replayed footprint touches (1,764 / 10,608).
5. **Bank sweep matches interleaving theory cell by cell.** Every one of the 20 measured cells matches what $\text{bank}(a) = a \bmod B$ with lowest-port-wins predicts. E.g. stride-4 at $B=4$: all four ports map to bank 0, so 3 of 4 stall every cycle $\rightarrow 3 \times 200 = 600$ stalled port-cycles. Measured: 600. $B=1$ is 0 by construction and measures 0.
6. **Directed and software test coverage.** 13 directed RTL tests (reset, each opcode in isolation, mixed phases, back-to-back phases, reset mid-operation, zero-op phases, busy handshaking, unknown-opcode detection, phase base addressing, full stream replay) with 48 passing assertions and zero failures; plus 27 software tests for the controller and 17 for banking.
7. **Regression safety.** The full 19/19 golden regression was re-run after *each* change (plumbing, testbench work, PT-write fix): all PASS every time, with the other members’ results byte-identical (tiny OS 5,808 cycles / 0.013%; WS 8×8 153,738; mnist OS 36,712 / 0.009%; all five schedulers `all_pass`). The PT-write broadcast cannot affect legacy runs because `pt_write_en` is never asserted there — confirmed by measurement, not just argument.

13.3 Internal consistency of the sweep

6 layers $\times$ 4 capacities $\times$ 2 arrays = 48 grid points; `sweep_stamp.csv` holds 34 rows and `capacity_limits.csv` holds 14 — they sum exactly to 48, so no point was silently dropped. Each failure records *why* (“Phase 0 doesn’t fit in on-chip memory”).

13.4 What would falsify the claims

- If the RTL counters diverged from the compiler’s, the scheme would not be implementable as described.
- If the bank sweep showed conflicts *independent* of the address pattern, static placement could not help and the central argument collapses.
- If the off-chip traffic reduction vanished at larger tile sizes, the reuse model would be wrong.

All three are directly testable with the harness as built — which is itself part of the evidence.

13.5 Credibility markers to point at

- **I report the results that go against me** — PAGED’s lower AXI beat count, STAMP’s higher cycle count, STAMP’s $4\times$ register cost and 32% lower Fmax.
- **I fixed the competitor’s bug before comparing.** The paged page-table write gate made ports 1–3 permanently miss; I found it, fixed it, and re-ran the entire comparison against the corrected, i.e. stronger, baseline.
- **I removed my own hardcoded numbers.** The previous four-scheme comparison was deleted, not defended, and the superseded outputs are kept clearly labelled so the change is auditable.
- **Every figure is traceable** to a JSON, CSV or synthesis report produced by a tool run, and `run_full_eval.py` contains no performance constants at all.
- **Conservative by construction** — the delta model under-claims reuse on diagonal shifts, and ZERO was added specifically to stop padding inflating the reported saving.

14 Rapid-fire answers to likely examiner questions

Q. What exactly is novel? Delta compression is not new.

A. Correct. The novelty is threefold: applying the delta to *scratchpad layout* in a systolic-array accelerator rather than to data values; eliminating the runtime tag/translation structure entirely by exploiting a statically known DNN schedule; and — what no prior simulator has — measuring the result cycle-accurately in RTL against a page-table scheme on identical hardware.

Q. Why not just use a bigger scratchpad?

A. Orthogonal and cheaper. A larger scratchpad costs area and leakage; delta fetching costs a compile-time-sized metadata RAM. And no scratchpad is large enough for real layers — the reuse problem persists at every size. The scheme raises the effective bandwidth of whatever scratchpad you have.

Q. Your paged scheme moves fewer AXI beats. Doesn't that defeat your claim?

A. On these layers, yes, and I report it. Page faulting pulls whole 4KB pages, which for a small working set is coarse but efficient. My claim is fewer bank conflicts ($42\times$), no translation hardware, and deterministic placement. For larger layers where a fault pulls far more than the delta, the byte comparison inverts — a falsifiable prediction.

Q. How do you know the compiler and the hardware agree?

A. Three independent implementations produce identical values for all eight tracked quantities across 572 operations, and the RTL reads the compiler's hex file directly with `stats_bad_ops = 0`.

Q. What does ZERO do and why does it exist?

A. Padded convolutions have receptive fields extending past the tensor border. Those bytes are implicit zeros: present on-chip, never fetched. Before the opcode existed the compiler charged them as DRAM loads, overstating traffic by 77,120 B. Adding it reduced the reported load volume from 235,008 to 157,888 B — it made the number *worse for the story and truer*.

Q. What is the hardware cost of your scheme?

A. A nine-state FSM plus the metadata RAM. The tag array, comparators and replacement logic disappear. Measured at 256 entries: 6,434 LUTs / 22,198 FFs at 405 MHz, versus 6,110 / 5,440 at 594 MHz for the paged manager — so similar LUTs, $4\times$ the registers, 32 % lower Fmax. The metadata RAM (9,152 B for the 128-phase layer) is the real cost and the main scaling limit.

Q. Why do the two schemes differ so much in bank conflicts?

A. The stamp compiler picks physical addresses at compile time and bump-allocates densely, so concurrent reads are stride-1 and land in different banks. The page table assigns physical pages at runtime and preserves page-relative offsets, so row-strided reads collide. The bank sweep proves conflicts are a deterministic function of the address pattern — exactly what static placement controls.

Q. Isn't your bank-conflict estimate in Experiment 3 just a formula?

A. Yes, and it is labelled as such. It is a birthday-problem bound used to cover 14 workloads where full RTL simulation is impractical. Experiment 4b is the authority — cycle-exact counters. The measured values sit far below the bound because real access patterns are consecutive, not random, and that gap is itself a finding.

Q. How would this behave on a multi-DNN workload?

A. Each DNN gets its own stamp schedule and the scratchpad is partitioned at compile time. Because placement is static there is no inter-DNN interference in a translation structure — an advantage over a shared page table needing multiple coexisting address spaces. Demonstrating it is future work.